Final Project – Robot Guidance Challenge

Technical Report

November 25, 2025

- Prepared for -

Osama Harmouche
Lab Instructor, COE538 – Microprocessor Systems

- Prepared by -

Kelin Mathew Jacob
501242848

Dev Brahmbhatt
501235383

Table of Contents

Project Overview

The final project for this course required students to consider aspects of the various labs encountered throughout the course, thus far. Specifically, the goal of this project was to configure the *'eebot'* to effectively traverse through a maze, from start to finish. Furthermore for additional functionality and bonus points, students were to also program the '*bot'* to reverse the maze from end to start after reaching the finish line. The maze in itself was to consist of two particular types of intersection, T-Junctions and L-Junctions, with the names being explanatory of the intersection characteristics.

T-Junction:

Bot comes to a stop and is at a point where:

- Sensor A does not read a line
- Sensor B and D both read lines

L-Junction

Bot comes to a stop and is at a point where:

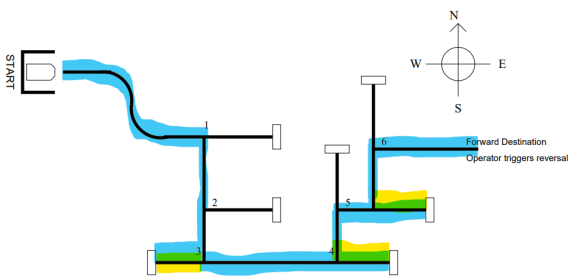
- Sensor A reads a line ahead
- Only one of sensor B/D reads a line ahead (left or straight / right or straight).

Ultimately to complete this project, individuals needed to keep various aspects of the challenge in mind, this included things like state control, intersection logic, port control etc. Hence, through completion of this challenge individuals will not only deepen their understanding of how microcontroller's work, but will have also learned how to tackle such cohesive and comprehensive challenges.

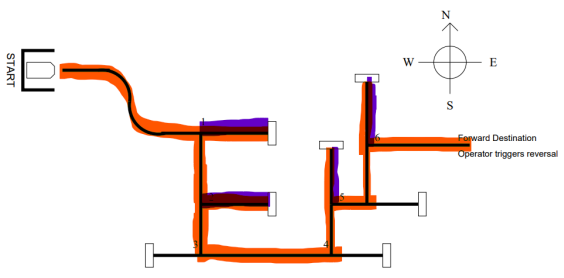
Our Approach

In order to tackle this project, we first mapped out potential solutions to the challenge at hand, and came to three viable solutions. We noted that the maze could be solved if the '*bot'* took right turns at T junctions, and L junctions where D was read. Else for L junction's where B would see a line, the Bot would go straight. By doing this the 'eebot' would effectively not take left turns. Consequently we noted that we could apply this same idea to left turns, in that the bot would only take left turns or go straight. Both these ideas would ensure effective completion of the maze, and the routes are shown below:

RIGHT ONLY (yellow represents u-turn path):



LEFT ONLY (purple represents u-turn path):



Finally the last solution we considered was one where the bot would take on any path in an intersection, and adjust to another path if the chosen one was a dead end. For instance at the very first turn on the map, the bot could go straight then as it returns, it remembers that the other option was right, and so this time it takes the opposite turn of that, ie… left. Eventually we realized that this logic would work for L-junctions, but would fail at T-junctions, given the tight deadline at hand, we decided to ultimately go forth with option 1 (right only).

Challenges Encountered & Overcome

While the solution we mapped out would solve the maze challenge and was simpler than trying both left and right turns, its implementation also posed various challenges.

Sensor Calibration:

When we initially completed our code we used random black/white backgrounds to determine the threshold values for the different sensors. When we then went to test our bot on the maze, it would work but not make turns properly, and sometimes it wouldn't even move. While the fix for this was simple - we needed to calibrate it to the tape, we didn't realize this immediately and spent quite a bit of time trying to troubleshoot. Eventually we realized this and we're able to calibrate the sensors accordingly, however even then oftentimes the sensors wouldn't perform as expected, leading us to believe there's some issue in the code, even when that wasn't the case.

Bot Availability:

This was not a technical, but a logistical challenge. We first considered testing on the *eebot* as we developed, however the issue with this is that we only have access to the bot during labs. Moreover, the possibility of facing an issue during the lab that would stop us from further testing was also undeniable. To overcome this we used a tactical approach that allowed us to optimize our time testing in the lab. We developed our program at home, and tried various different things, moreover when testing we tested part by part before attempting to see if the bot would solve the whole maze. For instance making sure it detects the line black lines as one part. Ultimately, through this tactical approach we were able to make the most of our time during the lab sessions.

Description of Code

Defining Values/Thresholds

```
;*****************************************
;* EQUATES SECTION
;*****************************************
LCD_DAT         EQU   PORTB
LCD_CNTR        EQU   PTJ
LCD_E           EQU   $80
LCD_RS          EQU   $40

;Movement/Threshold variables - vary by bot
;-----------------------------------------
PTH_A_INT       EQU   $C0
;PTH_B_INT       EQU   $CA
PTH_C_INT       EQU   $CA
PTH_D_INT       EQU   $CA

PTH_E_INT       EQU   $75
PTH_F_INT       EQU   $75

INC_DIS         EQU   300
FWD_DIS         EQU   400
REV_DIS         EQU   1000
STR_DIS         EQU   1000
TRN_DIS         EQU   10000
UTRN_DIS        EQU   12000
;-----------------------------------------

PRI_PTH_INT     EQU   0
SEC_PTH_INT     EQU   1

; defining the various states
START           EQU   0
FWD             EQU   1
REV             EQU   2
RT_TRN          EQU   3
LT_TRN          EQU   4
BK_TRK          EQU   5
SBY             EQU   6
```

Various different values for the sensors and the bot's movement that are used throughout the code are defined at the start of the program, as seen in the snippet above. The PTH_A_INT, PTH_B_INT… all store the threshold values for the respective sensors to see a path or not. The Distances define the amount that the bot must turn/move before it re-evaluates, all of these values vary based on the *eebot* being used. Near the end, the varying states represent the modes that the *eebot* goes through as it traverses through a path. These constants are used in the dispatcher to effectively control/determine the *eebot's* state.

Motor Speed Control

Adjusting how fast the *eebot* traversed the maze was crucial since we needed to choose an appropriate speed for the sensors to collect information properly. If the *eebot* moves too fast, the signals from the sensors would be corrupted and become less accurate. Since the motors always run at full voltage, we took a different approach where we would control the bot's *effective speed* by counting timer interrupts:

```
;*******************************************************************************************
;* INTERRUPT SERVICE ROUTINE 1                                                             *
;*******************************************************************************************
ISR1            MOVB  #$01,TFLG1                ; clear the C0F input capture flag
                INC   COUNT1                    ; increment COUNT1
                RTI
;*******************************************************************************************
;* INTERRUPT SERVICE ROUTINE 2                                                             *
;*******************************************************************************************
ISR2            MOVB  #$02,TFLG1                ; clear the C1F input capture flag
                INC   COUNT2                    ; increment COUNT2
                RTI
```

These counters essentially determine how long the motors run by comparing COUNT1/COUNT2 to thresholds. For simplicity, we will take the example when the bot is moving straight:

```
FWD_STR_DIS     LDD   COUNT1                ;
                CPD   #FWD_DIS               ;
                BLO   FWD_STR_DIS            ; If Dc>Dfwd then
                JSR   INIT_FWD               ; Turn motors off
```

Here, the robot drives forward until COUNT1 > FWD_DIS. If we increase FWD_DIS (shown below), the robot drives forward for more time per loop, *effectively* moving slower because the robot spends more time "waiting," state transitions happen slower, and the robot completes straight segments more slowly.

```
FWD_DIS         EQU   400                  ; Constant forward distance
REV_DIS         EQU   1000                 ; ........ reverse distance
STR_DIS         EQU   1000                 ; ........ distance to travel after turn, to reposition
TRN_DIS         EQU   10000                ; ........ 90 degree turn distance
UTRN_DIS        EQU   12000                ; ........ complete u-turn distance
```

The same applies to turn distances. Increasing these values effectively results in the *eebot* moving slower and vice-versa.

Collecting Sensor Data

This is the main routine for reading the sensors:

```
UPDT_READING    JSR   G_LEDS_ON            ; Turn on the leds
                JSR   READ_SENSORS           ; Take sensor readings
                JSR   G_LEDS_OFF             ; Turn off the leds
```

Since the sensors require LED illumination for reflective sensing, bit 5 of Port A is used to turn on/off the LEDs:

```
;*******************************************************************************************
G_LEDS_ON       BSET  PORTA,%00100000        ; sets bit 5
                RTS
;*******************************************************************************************
G_LEDS_OFF      BCLR  PORTA,%00100000        ; clears bit 5
                RTS
```

The sensor numbers are assigned as follows:

0 → E/F (line)
1 → A (bow/front)
2 → B (port/left)
3 → C (middle)
4 → D (starboard)
A hardware multiplexer selects the appropriate sensor for reading (starts at 0, ends at 4). In each iteration of the loop, a 20ms delay is used to allow the photoresistors to settle (since they respond slowly to sudden changes in illumination) and the A/D conversion of the raw sensor values is triggered.

```
READ_SENSORS    CLR   SENSOR_NUM              ; Start with sensor number 0
                LDX   #SENSOR_LINE            ; Point at the start of the sensor array

  RS_MAIN_LOOP: LDAA  SENSOR_NUM              ; Choose correct sensor input
                JSR   SELECT_SENSOR           ; on the hardware

                LDY   #200                    ; sensor stabilization takes 20 ms delay
                JSR   del_50us

                LDAA  #%10000001              ; A/D conversion on AN1 begins here
                STAA  ATDCTL5
                BRCLR ATDSTAT0,$80,*          ; Repeat until A/D signals done
                LDAA  ATDDR0L                 ; A/D conversion is complete in ATDDR0L
                STAA  0,X                     ; so copy it to the sensor register

                CPX   #SENSOR_STBD            ; Exit if this is the last reading
                BEQ   RS_EXIT
                INC   SENSOR_NUM              ; otherwise increment sensor #
                INX                           ; and the pointer into the sensor array
                BRA   RS_MAIN_LOOP            ; repeat for remaining sensors
       RS_EXIT: RTS
```

Processing Sensor Data

For simplicity, we will show the data processing for sensor A (since A, B, C, and D have the same functionality → pattern detectors), and E/F (line tracking).

Each sensor is compared to a threshold. For example, for sensor A, if its value is greater than/equal to the threshold, that means there is a path detected (otherwise no path).

```
CHECK_A         LDAA  SENSOR_BOW              ; If SENSOR_BOW is > than
                CMPA  #PTH_A_INT              ; PTH_A_INT
                BLO   CHECK_B                 ;
                INC   A_DETN                  ; Set A_DETN = 1
```

For the line tracking sensors, if the value of sensor E is less than the threshold, it means that the bot is too far right and needs to shift left; if the value of sensor F is greater than the threshold, it means the bot is too far left and needs to shift right. Essentially:

E_DETN = 1 → move left
F_DETN = 1 → means move right

```
CHECK_E         LDAA  SENSOR_LINE             ; If SENSOR_LINE is < than
                CMPA  #PTH_E_INT              ; PTH_E_INT
                BHI   CHECK_F                 ;
                INC   E_DETN                  ; Set E_DETN = 1

CHECK_F         LDAA  SENSOR_LINE             ; If SENSOR_LINE is > than
                CMPA  #PTH_F_INT              ; PTH_F_INT
                BLO   UPDT_DONE               ;
                INC   F_DETN                  ; Set F_DETN = 1
```

Dispatcher & Intersection Detection

We can understand how the dispatcher works by only considering the following cases, as the logic repeats

```
;*****************************************************************************************
;* DISPATCHER - STATES SECTION
;*****************************************************************************************
DISPATCHER      CMPA  #START                    ; Checks if in start state -----------------
                BNE   NOT_START                 ; if not goes onto next state check
                JSR   START_ST                  ; else if it is then calls START_ST routine
                RTS                             ; exits
                                                ;
NOT_START       CMPA  #FWD                      ; Checks if its the FORWARD state
                BNE   NOT_FORWARD               ; if not goes onto next state check
                JMP   FWD_ST                    ; else if it is then calls FWD_ST routine
```

The value of the current state is stored in accumulator A, CMPA #START compares the value in AccA with #Start, if they are the same then we jump to the START_ST routine, else it checks the value in AccA with the other states, until a match is found. This process repeats for all the states noted previously, and is analogous to the dispatcher code we have considered in previous labs.

Let us consider what happens as we approach a right or straight intersection while in the FWD state.

```
FWD_ST          PULD                            ;
                BRSET PORTAD0,$04,NO_FWD_BUMP   ; If forward bump is not hit, go into NO_FWD_BUMP, else go to next line
                LDAA  SEC_PTH_INT               ;
                STAA  NEXT_D                    ;
                JSR   INIT_REV                  ; if the fwd bump is hit, this means we encountered obstacle, must turn around: Initialize the REVERSI
                MOVB  #REV,CRNT_STATE           ; CRNT_STATE to REV
                JMP   FWD_EXIT                  ; and return

NO_FWD_BUMP     BRSET PORTAD0,$08,NO_REV_BUMP   ; If the rev bump has also not been hit, go to NO_REV_BUMP, else continue
                JMP   INIT_BK_TRK               ; if the back bumper is hit, backtrack.
                MOVB  #BK_TRK_CRNT_STATE        ;
                JMP   FWD_EXIT                  ; and return

NO_REV_BUMP     LDAA  D_DETN                    ; If neither bumper is hit, check if there is a line on the right.
                BEQ   NO_RT_INTXN               ; If not, then Z = 1 (Z is 0 for nonzero number, 1 or zero), and go to NO_RT_INTXN
                LDAA  NEXT_D                    ; else: if there is a line on the right, we will take the turn.
                PSHA                            ;
                LDAA  PRI_PTH_INT               ;
                STAA  NEXT_D                    ;
                JSR   INIT_RT_TRN               ; Initialize the RT_TRN state
                MOVB  #RT_TRN,CRNT_STATE        ; Set CRNT_STATE to RT_TRN
                JMP   FWD_EXIT                  ; Then exit
```

We first check if the fwd/rev bumps have been pressed, by using BRSET. Given we are approaching a right turn, this is not the case, so we jump to the NO_REV_BUMP routine. Within here, we now load the value in D_DETN into AccA. If there is a path to the right, then D_DETN = 1, hence Z = 0 and we continue. After which the Right turn states are initialized and the current state is changed. Once the right turn will be completed this procedure of checking will once again take place, but this time from within the RT_TRN routine. Covering all the cases for intersection turns within this report would be tedious, however the idea is analogous to what we have seen in this case. The bumpers are checked, the sensors are checked, based on priority the bot goes straight or takes a turn.

Conclusion

Overall this project was both a success, and a very big learning opportunity. Through this challenge, we were able to experience what it feels like to take on a large scale hardware/software project, in a team environment. In particular, this experience helped deepen our understanding of the process and project management aspects that go into tackling such a project. In the end, our bot was able to successfully complete the maze, from start to finish, using the right/straight only approach, however we didn't have enough time to complete the additional functionality of path reversal. While this may seem like a let down, it has helped us understand the importance of planning, accounting for challenges, and also cutting back on requirements to meet deadlines. Ultimately despite the numerous challenges faced in regards to sensor calibration, bot availability, testing, and communicating amongst ourselves, we were able to successfully complete the maze, all the while furtherering our understanding of testing, hardware validation/verification and logical analysis.